

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Петрозаводский государственный университет»

Институт математики и информационных технологий
кафедра ИМО

Пянтин Артём Сергеевич, группа 22406
Титов Евгений Викторович, группа 22405

РАЗВИТИЕ СИСТЕМЫ «КУРС»

Направление подготовки бакалавра
09.03.02 — Информационные системы и технологии
Форма обучения — очная

Допущена к защите.

Заведующий кафедрой,

к.т.н., доцент

Богоявленский Юрий Анатольевич

«___» _____ 2026_ г.

Руководитель

доцент

Кулаков Кирилл Александрович

«___» _____ 2026_ г.

старший преподаватель

Чистяков Дмитрий Борисович

«___» _____ 2026_ г.

Петрозаводск — 2026

Оглавление

Введение	3
1 Исходная система и теоретические основания	5
1.1 Обзор исходной системы	5
1.2 Обзор аналогичных систем в вузах и колледжах	6
1.3 Обзор Clojure как исходной технологической базы	9
1.4 Сравнительный анализ Clojure и Python	9
1.5 Выбор фреймворка и архитектурный подход	11
1.6 Качество, риски и ограничения	12
1.7 Итоговые требования к результату	13
1.8 Требования к расширению функциональности	13
1.9 Общее описание системы	14
1.10 Требования и сценарии использования	14
2 Реализация и тестирование модернизации	17
2.1 Исходная архитектура и база данных	17
2.2 Изменения в проектировании	18
2.3 Общая архитектура данных	19
2.4 Диаграмма связей между таблицами	19
2.5 Описание основных таблиц	20
2.6 Реализация функциональности и принятые решения	20
3 Результаты модернизации	27
3.1 Сравнение с предыдущей версией системы	27
3.2 Метрики выполненной работы	31
Заключение	33
Библиографический список	34

Введение

Область разработки — прикладные веб-информационные системы, поддерживающие учебный процесс в вузе: учёт тем и документов, разграничение доступа, согласование действий преподавателя и студента, сохранение истории на несколько учебных лет [2, 1]. С позиции инженерии программного обеспечения такие системы ближе к корпоративным приложениям с периодами пиковой нагрузки, чем к разовым лабораторным прототипам: ошибки в регламенте или в данных быстро превращаются в организационные сбои, поэтому важны целостность модели, прозрачность сценариев и устойчивость сопровождения [1].

Система «КУРС» — действующий сервис кафедры: в ней регистрируют темы, загружают материалы и отслеживают статус работы [16]. Изначально была реализована на Clojure/Ring [18, 3].

По функциональному охвату «КУРС» близка не корпоративным ЕСМ/СЭД, а специализированным сервисам вузов и колледжей, где ведут учёт курсовых и выпускных работ: кафедральные веб-системы, университетские порталы сдачи ВКР, электронные репозитории. Отличие «КУРС» в том, что сервис заточен под регламент конкретной кафедры, сопоставление с отечественными аналогами приведено в разделе 1.2 [16, 7, 2].

Возможные направления развития исходного решения можно свести к четырём группам: (1) эволюция файловой модели при сохранении стека Clojure и доработке парсеров; (2) перенос хранения в реляционную СУБД без смены языка реализации; (3) переход на распространённый веб-стек с миграцией данных и перепроектированием процессов; (4) замена собственной разработки коммерческим или открытым LMS с последующей интеграцией. Варианты (1) и (2) сохраняли бы узкий кадровый контур; вариант (4) дал бы больше готовых модулей, но ухудшил бы управляемость локальными правилами и стоимость сопровождения доработок. Мы выбрали вариант (3): он сочетает преемственность сценариев с реляционной формализацией данных и более доступным для университетской команды циклом разработки [4, 5, 1, 20, 19].

Исходная версия оставалась работоспособной, но со временем стала ограничивать развитие: файловая модель плохо подходила для расширения ролей, заявок и статусов. Мы решали задачу не полным «обнулением» кодовой базы, а последовательной актуализацией: формализовали данные в реляционной схеме [5, 4], переработали ключевые сценарии и добавили процессы, которых не хватало в ежедневной работе.

Цель работы — актуализация системы «КУРС» и добавление функционала, необходимого для устойчивого сопровождения полного учебного цикла (тема, заявка, документы,

коммуникация, отчётность).

Для достижения цели решались следующие задачи:

1. Выполнить анализ исходной технологической базы, включая рассмотрение Clojure как языка реализации существующего решения.
2. Провести сравнительный анализ языков и фреймворков (Clojure/Python, Flask/Django) и обосновать выбор стека дальнейшей разработки.
3. Спроектировать архитектуру приложения и структуру базы данных с учётом требований к расширяемости и целостности данных.
4. Реализовать функциональные изменения в пользовательских и административных сценариях системы.
5. Провести позитивное покрывающее тестирование на тестовой среде и зафиксировать результаты выполнения тестовых сценариев.
6. Сопоставить текущую версию системы с предыдущей реализацией и оценить достигнутый эффект модернизации.

Отчёт собран в три крупных блока: что представляла собой исходная система, какие изменения мы внесли и какие результаты получили после внедрения.

При подготовке текста отдельных разделов отчёта использовались средства искусственного интеллекта для уточнения формулировок с последующей редакторской обработкой и проверкой содержания авторами.

Глава 1

Исходная система и теоретические основания

1.1 Обзор исходной системы

Исходная версия системы (предыдущая версия) представляла собой веб-приложение для учёта курсовых и выпускных работ кафедры. Система решала практическую задачу: дать студентам и преподавателям единый интерфейс для регистрации данных о работе, загрузки PDF-документов и поиска материалов по учебным годам и группам [16]. С архитектурной точки зрения это был серверный монолит на связке Ring/Compojure с шаблонным интерфейсом и LDAP-аутентификацией [18].

Ключевая особенность реализации заключалась в файловой модели хранения, когда целостность опирается на соглашения о структуре файлов, а не на декларативные ограничения СУБД [4, 5]. В качестве «базы данных» использовалась иерархия каталогов вида «год — курс — группа — логин», а в каталоге студента хранился индексный файл с метаданными о работе. Индекс мог иметь форматы XML, JSON или EDN/CLJ; из него извлекались ФИО, тема, сведения о руководителе, кафедра и служебные отметки времени. Таким образом, логика целостности была сосредоточена не в реляционных ограничениях, а в правилах парсинга и структуре файлового архива.

Серверная часть содержала полный набор маршрутов для ежедневной работы. Были реализованы: главная страница со списком работ, формы поиска и фильтрации, страница логина, личный кабинет студента, страница загрузки/удаления документов, а также специализированный маршрут преподавателя со списком своих студентов. Отдельные POST-маршруты формировали печатные отчёты (PDF), список незарегистрированных студентов и распоряжения в формате RTF. За счёт этого система закрывала не только хранение данных, но и типовые отчётные операции кафедры в учебный период.

Механизм аутентификации строился вокруг LDAP: система искала пользователя в каталоге, проверяла пароль и определяла роль по атрибутам учётной записи. Для студентов при первом входе автоматически инициализировалась каталоговая структура текущего учебного года. Также существовал тестовый контур входа с сервисными учётными запи-

сями и «мастер-паролем» для администрирования и аварийных сценариев, когда LDAP недоступен.

Подсистема работы с файлами поддерживала полный цикл операций: сохранение и удаление документов, проверку формата (разрешался PDF), копирование данных из предыдущего года и обновление индекса после изменений. Дополнительно использовался кэш результатов парсинга с файловым наблюдателем: при изменении каталогов кэш сбрасывался, и данные пересчитывались. Это ускоряло выборки при частых запросах, но одновременно усиливало зависимость всей системы от корректности файловой структуры.

С точки зрения пользовательских возможностей исходная версия обеспечивала устойчивый базовый функционал: регистрацию темы и руководителя, загрузку промежуточных и итоговых материалов, поиск по параметрам, просмотр карточек работ и генерацию печатных форм. Именно этот функциональный фундамент сделал возможной дальнейшую модернизацию: при переходе на новую версию мы сохраняли рабочие сценарии, но переносили их из файловой парадигмы в формализованную модель данных и расширяли ролевые процессы.

1.2 Обзор аналогичных систем в вузах и колледжах

Под «документооборотом» курсовых и выпускных работ в высшей школе обычно понимают не корпоративные СЭД, а прикладные сервисы, в которых студент передаёт файлы, а кафедра фиксирует тему, этапы и результаты [2, 3]. В российской практике такие решения реализуются на трёх уровнях: *кафедральные* (узкий контур одной специальности), *университетские* (единый сервис или репозиторий для всего вуза) и *колледжские* (модули комплексных АИС СПО). «КУРС» относится к первому типу; для обоснования модернизации мы сопоставили её с публично описанными аналогами других организаций [11, 12, 13, 14, 15].

Кафедральные и факультетские системы (пример: ТПУ). Для иллюстрации кафедрального уровня можно привести информационную систему учёта и контроля этапов подготовки ВКР, разработанную на кафедре информационных систем ЮТИ Томского политехнического университета [11]. В опубликованном описании акцент сделан на календарно-этапном контроле: задаётся график выполнения, фиксируются статусы этапов, формируется сводная аналитика по группе и по студентам. С практической точки зрения такая система закрывает задачу дисциплины выполнения и прозрачности прогресса (что/когда сдано, где возникли задержки), но не обязательно включает подробный контур выбора темы через заявки и последующую коммуникацию в рамках одной платформы. По назначению это ближайший аналог «КУРС» как локальная разработка под регламент подразделения, а не покупной корпоративный продукт: обе системы ориентированы на ежедневные действия учебного цикла и на хранение «живого» состояния процесса.

Университетские сервисы и репозитории (пример: ПГГПУ). В Пермском государственном гуманитарно-педагогическом университете эксплуатируется сервис «Выпуск-

ные квалификационные работы» [12]. По доступным материалам он относится к вузовому уровню: основной сценарий связан с централизованным сопровождением материалов ВКР и их публикацией/учётом в рамках университета. Как правило, в таких решениях приоритетом является унификация требований (состав пакета документов, корректность загрузки, централизованная отчётность) и обеспечение прозрачности для администрации вуза, при этом «тонкие» кафедральные правила выбора темы и согласования часто остаются за пределами системы или реализуются частично.

Вузовский контур передачи в библиотеку (пример: СПбПУ). В Санкт-Петербургском политехническом университете Петра Великого описан отдельный контур передачи ВКР и научных докладов в научную библиотеку [13]. Такой сценарий характерен для этапа, близкого к завершению работы: документы передаются в библиотечный фонд, выполняются требования по оформлению и сдаче, формируется архив для последующего хранения и доступа. Сильная сторона подхода — институциональное архивирование и контроль наличия материалов в едином месте; слабая по отношению к «КУРС» — отсутствие ежедневного «кафедрального конвейера» (каталог тем, заявки, оперативные статусы, обсуждение).

Каталоги в электронно-библиотечных системах (пример: ВолГУ). Волгоградский государственный университет ведёт каталог ВКР в электронно-библиотечной системе с поиском по автору, теме, году и направлению [14]. Это типичный пример репозитория: основной функционал связан с хранением и поиском по уже завершённым работам, поддержкой доступа и формированием витрины архива. Для задач учебного процесса такие системы полезны как «долгая память» университета, но они не предназначены для сопровождения стадий до защиты (выбор темы, обработка заявки, промежуточные документы, переписка).

Публикация сведений на уровне подразделения (пример: ЮФУ). На Южном федеральном университете институты публикуют сведения о размещении ВКР на сайте подразделения [15]. Этот уровень обычно даёт пользователю информацию о правилах, местах размещения и регламенте, а не полноценный интерактивный контур работы с темами и документами. В сравнении с «КУРС» такое решение выполняет информационную функцию и может быть частью общего процесса, но не замещает кафедральную систему управления этапами и статусами.

Колледжи (СПО) и комплексные АИС. В среднем профессиональном образовании учёт выпускных работ чаще встроен в комплексные платформы («1С:Колледж», E-SPO и аналоги) и связан с общим контуром учебного процесса: журналы, успеваемость, практики, приказы и выпуск документов об образовании [2]. В таких системах «выпускные материалы» обычно рассматриваются как один из модулей, а не как отдельный специализированный веб-сервис кафедры с каталогом тем, заявками и коммуникацией. Поэтому прямых аналогов «КУРС» на уровне СПО меньше; сопоставление с ними носит характер определения границ применимости и понимания того, какие функции целесообразнее оставлять в специализированном контуре.

Базовый функционал «КУРС» (исходная версия) для сопоставления: LDAP-вход; единый интерфейс для регистрации сведений о работе (тема, руководитель, служебные поля); загрузка и удаление PDF-документов; поиск и фильтрация по учебным годам и группам; карточки работ; формирование печатных форм и отчётных документов (PDF/RTF) для типовых операций кафедры [16, 18, 3]. При этом выделенный процесс «каталог тем — заявка — решение» и встроенный контур обсуждений по теме в исходной версии отсутствовали (они добавлены в ходе модернизации и рассматриваются далее в главе 3).

Содержательное сопоставление показало, что решения различаются прежде всего уровнем внедрения и типом закрываемого контура. Кафедральные системы ориентированы на ежедневные действия преподавателя и студента и отражают регламент подразделения: исходная версия «КУРС» обеспечивала учёт работ и документов (регистрация, загрузка, поиск, печатные формы), а ИС этапов ВКР в ТПУ делала акцент на формальном контроле графика этапов и аналитике. Университетские сервисы (ПГПУ, СПбПУ, ВолГУ, ЮФУ) чаще выполняют роль сдачи и архивирования материалов ВКР и поддержки общеузовской отчётности, но обычно не заменяют кафедральный цикл выбора темы и сопровождения работы в течение семестра. Для СПО типично использование комплексных АИС, в которых выпускные материалы являются частью общего модуля учебного процесса.

Для удобства чтения ключевые позиции сравнения сведены в компактную таблицу 1.1.

Таблица 1.1: Сопоставление исходной версии «КУРС» и аналогичных решений (по назначению)

Решение (организация)	Уровень	Основной контур
«КУРС» (ПетрГУ, исходная версия)	Кафедра	Учёт работ и документов: регистрация, поиск, загрузка PDF, отчёты/печатные формы; без процесса заявок и обсуждений
ИС этапов ВКР (ТПУ)	Кафедра	Контроль графика этапов и статусов подготовки; аналитика по группе
Сервисы/порталы ВКР (ПГПУ, СПбПУ, ВолГУ, ЮФУ)	Вуз/институт	Приём и публикация/архивирование материалов; поддержка отчётности на уровне вуза
АИС СПО (типовые решения)	Колледж	Учёт учебного процесса в целом; выпускные материалы как часть общего контура

По таблице видно: у российских вузов распространены репозитории и порталы сдачи ВКР, а кафедральные системы, закрывающие *ежедневный процессный контур* (выбор темы, согласование, сопровождение документов и коммуникация), встречаются реже. Исходная версия «КУРС» относилась к классу систем учёта и архивирования материалов на уровне кафедры; модернизация расширила её до процессной модели (см. главу 3), что повышает прикладную ценность по сравнению с решениями, ориентированными только на сдачу и архив. Поэтому развитие «КУРС» оправдано как специализированное решение кафедры с возможностью обмена данными с общеузовскими процессами, а не как замена

отечественной ЕСМ-платформы [7, 16].

1.3 Обзор Clojure как исходной технологической базы

Мы рассматривали Clojure не как «старый язык, который нужно заменить», а как осмысленную техническую основу предыдущей версии. Он хорошо подходит для серверных задач, где важны предсказуемое поведение и аккуратная работа с состоянием данных. Для кафедры это действительно важно: в период массовых сдач любая неочевидная ошибка быстро превращается в организационную проблему.

Функциональный стиль дал первой реализации ясную модель преобразования данных: входящие запросы рассматриваются как вход в чистые процедуры, а итог формируется из последовательных вычислений. Такой подход снижает риски, связанные с неконтролируемым изменением общей памяти, и делает поведение многих операций воспроизводимым. В то же время подобная архитектура требует от новых разработчиков специальной подготовки, поскольку синтаксис и идиомы Clojure заметно отличаются от массовых языков веб-разработки.

Особое значение имела интеграция с экосистемой Java. За счёт JVM проект получал доступ к зрелым библиотекам и проверенной инфраструктуре запуска. С организационной точки зрения это давало устойчивость, но усложняло вход для студентов-разработчиков, которым проще начинать с более распространённых инструментов. В учебной среде баланс между технической выразительностью и доступностью команды имеет принципиальное значение: система должна быть не только корректной, но и поддерживаемой на горизонте нескольких лет.

Проведённый анализ показал, что Clojure обеспечивает сильную основу для надёжной серверной логики, однако сопровождаемость и кадровая доступность становятся ограничивающим фактором для долгосрочного развития в университете. Поэтому в первой части проекта обзор Clojure был необходим не как критика предыдущего решения, а как корректное обоснование перехода к стеку, который проще масштабировать силами студенческой команды без потери качества.

При анализе исходной версии мы также учитывали фактическое назначение системы и её рабочий контур эксплуатации на кафедре [16].

1.4 Сравнительный анализ Clojure и Python

Сравнение языков проводилось применительно к задачам системы «КУРС», а не в отрыве от предметной области [1]. Главным критерием выступала стоимость развития: сколько усилий требуется, чтобы вносить изменения в сценарии работы преподавателей и студентов, поддерживать тестовый контур и быстро исправлять ошибки в период активной нагрузки. По этому критерию Python показал лучшую практическую применимость благодаря более низкому порогу входа и обширной экосистеме [19, 18].

С точки зрения архитектурной выразительности Clojure сильнее в задачах, где критичны функциональные преобразования и строгая работа с неизменяемыми структурами. Python, в свою очередь, предоставляет мультипарадигменный подход: в одном проекте можно комбинировать процедурные, объектные и частично функциональные приёмы, выбирая стиль под конкретный модуль. Для университетской системы с разнотипными задачами (формы, отчёты, ролевые сценарии, обмен сообщениями, администрирование) такая гибкость оказалась преимуществом.

Отдельно оценивалась доступность инженерных практик. Для Python существуют массовые учебные материалы, примеры архитектур, шаблоны тестирования и готовые интеграции с инструментами CI/CD. Это сокращает цикл внедрения новых функций и помогает держать единый стиль кода в распределённой команде. Для Clojure качественные материалы также есть, но их меньше, а скорость адаптации новых участников заметно ниже.

По критерию производительности на прикладном уровне проекта различия не стали определяющими. В «КУРС» узким местом чаще является не вычислительная сложность, а согласованность данных, корректная маршрутизация пользовательских сценариев и скорость доработки интерфейса. В этом контексте выигрыш Python проявился именно в темпе эволюции системы и удобстве сопровождения, что напрямую влияет на качество сервиса в учебном процессе.

Если подвести итог кратко: Clojure оказался надёжной технической базой, но для нашей команды и формата университетской поддержки Python дал более удобный и быстрый путь развития. Поэтому именно на Python мы продолжили проект и реализовали расширенный функционал.

В процессе сравнения мы не опирались только на общие рассуждения про популярность языков. Мы проверяли, как быстро можно внести небольшое изменение в рабочий сценарий и безопасно довести его до тестового запуска. На Python этот цикл оказался заметно короче: проще читать код, проще подключать новые модули и проще согласовывать правки в команде. Для учебного проекта, где сроки и ресурсы ограничены, это стало решающим аргументом.

Ключевые различия языков для задач проекта приведены в таблице 1.2.

Таблица 1.2: Сравнение Clojure и Python для задач системы «КУРС»

Критерий	Clojure	Python
Порог входа в проект	Требуется знание функциональной парадигмы и специфического синтаксиса	Ниже за счёт массового распространения и понятного синтаксиса
Скорость включения новых разработчиков	Умеренная, требуется дополнительная адаптация	Высокая, доступно много учебных материалов и практик
Гибкость прикладной разработки	Сильна в функциональных преобразованиях	Удобна для смешанных сценариев: ORM, веб, отчёты, интеграции
Экосистема библиотек	Устойчивая экосистема JVM	Широкая экосистема для веб-разработки и автоматизации
Поддержка проекта в учебной среде	Эффективна при подготовленной команде	Предпочтительна для длительного сопровождения сменяемой командой

1.5 Выбор фреймворка и архитектурный подход

После выбора языка был выполнен анализ веб-фреймворков с учётом ожидаемого жизненного цикла проекта [1, 7, 8]. Рассматривались два базовых подхода: минималистичный каркас с ручной сборкой компонентов и полнофункциональная платформа с готовыми механизмами безопасности, аутентификации, ORM и административного интерфейса. Для системы, где важно быстро перейти от теории к устойчивой эксплуатации, приоритет был отдан второму варианту.

Django был выбран как технологическая основа первой части по совокупности факторов [20, 7]. Во-первых, встроенная ORM позволила сразу задать устойчивую модель данных и сократить объём инфраструктурного кода [5]. Во-вторых, встроенные механизмы авторизации и разграничения доступа снизили риски, связанные с ручной реализацией критичных аспектов безопасности. В-третьих, административная панель ускорила сопровождение справочников и помогла организовать контроль данных без разработки отдельного back-office интерфейса.

Альтернативный микрофреймворковый подход (например, Flask) предоставлял большую свободу настройки, но требовал самостоятельной сборки значительного числа компонентов: управления доступом, структуры проекта, админского контура и части защитных механизмов. Для учебной команды это означало бы дополнительную нагрузку на поддержку инфраструктуры вместо развития предметного функционала.

Поэтому в первой части проекта архитектурное решение формулировалось как «практически ориентированный full-stack подход»: фреймворк должен закрывать типовые инженерные задачи из коробки, а команда должна сосредоточиться на бизнес-логике системы «КУРС». Этот выбор подтвердился на следующем этапе, когда уже на готовом каркасе удалось без радикальной перестройки внедрить темы, заявки, административные

сценарии и realtime-коммуникацию.

Сводное сравнение фреймворков, использованное при принятии решения, представлено в таблице 1.3.

Таблица 1.3: Сравнение фреймворков Django и Flask

Критерий	Django	Flask
Подход к разработке	Полнофункциональный фреймворк	Минималистичный каркас
Готовые механизмы из коробки	ORM, админка, аутентификация, маршрутизация, формы	Базовая маршрутизация, остальное подключается отдельно
Скорость старта для среднего проекта	Выше за счёт встроенных компонентов	Ниже из-за необходимости сборки инфраструктуры вручную
Управляемость при развитии функционала	Высокая при соблюдении архитектурной дисциплины	Зависит от качества локальных соглашений и структуры проекта
Соответствие задачам «КУРС»	Подходит для ролевой системы и административного сопровождения	Требуется дополнительной реализации системных компонентов

1.6 Качество, риски и ограничения

Помимо технологического выбора, в первой части отдельно оценивались риски внедрения [1]. Наиболее существенными считались четыре группы: риск неполной функциональной преемственности, риск рассогласования данных при миграции, риск роста трудоёмкости сопровождения и риск организационных задержек при вводе новой версии в практическую работу. По каждому риску фиксировались меры снижения: поэтапный перенос сценариев, контрольные выборки данных, формализация модели ролей и постепенное внедрение через тестовый контур.

Отдельный блок требований касался качества пользовательского взаимодействия. Даже на базовом этапе система должна была предоставлять предсказуемый интерфейс: явные формы ввода, понятную реакцию на действия пользователя, корректные сообщения о результате операций и сохранение связности навигации между разделами. Для учебного сервиса эти свойства имеют прикладной смысл: пользователь не должен тратить время на интерпретацию состояния системы в период подготовки отчётных материалов.

С инженерной точки зрения требовалась управляемая модульность [7, 10]. Компоненты, связанные с данными, представлениями и служебными процедурами сопровождения, проектировались так, чтобы их можно было расширять независимо. Это позволило не только стабилизировать первую часть проекта, но и подготовить основу для второй части, где объём изменений значительно выше и требует более строгой структуры кода и ответственности модулей.

Наконец, существенным ограничением был кадровый фактор: в студенческой команде

состав участников меняется, а длительные паузы между этапами неизбежны. Поэтому решения первой части принимались с приоритетом на читаемость и повторяемость: система должна быть понятной не только её авторам, но и новой команде, которая продолжит развитие без потери контекста. Этот принцип стал определяющим при оформлении документации, структуре модулей и выборе распространённого стека.

1.7 Итоговые требования к результату

Итоговый результат первой части рассматривался не как конечный продукт, а как устойчивая платформа для дальнейшего наращивания функциональности. Минимальный набор требований включал корректную работу с учётными данными, хранение учебных связей, приём и выдачу документов, административное сопровождение справочников и возможность воспроизводимого развёртывания в тестовом контуре.

Качественным критерием считалась способность системы выдерживать типовую нагрузку учебного периода без ручной доработки данных [4, 1]. Это требовало обеспечения целостности связей на уровне модели, аккуратной работы с файлами и единых правил формирования отчётов. Дополнительно было важно, чтобы дальнейшие изменения можно было вводить инкрементально: добавление нового процесса не должно нарушать уже работающие сценарии.

Таким образом, первая часть проекта выполнила роль методологической и инженерной базы. Она обеспечила формальный переход на новый стек, зафиксировала обоснование ключевых решений и создала структуру, в которой вторая часть смогла сосредоточиться на развитии прикладного функционала, а не на повторном решении инфраструктурных проблем.

1.8 Требования к расширению функциональности

Во второй части ключевым требованием стало не повторение переноса, а расширение уже действующей системы под реальные процессы взаимодействия преподавателя и студента в течение учебного цикла. Добавляемый функционал должен был обеспечить управляемый каталог тем, прозрачный процесс подачи и рассмотрения заявок, ролевую навигацию, встроенные обсуждения и инструменты сопровождения на уровне кафедры.

Дополнительно были зафиксированы требования к качеству эксплуатации: воспроизводимое развёртывание в контейнерной среде, поддержка импорта данных и сохранение согласованности между базой и файловым архивом. Именно эти требования определили дальнейшие решения в проектировании и реализации.

1.9 Общее описание системы

«КУРС» — это Django-приложение, закрывающее полный контур «тема–заявка–документы»: от выставления темы руководителем до выбора студентом, согласования и складывания материалов в хранилище. В модели участников выделено три типа:

- студент видит каталог, отправляет заявку, следит за решением и прикладывает файлы;
- преподаватель формирует и правит темы, разбирает очередь заявок;
- администратор поддерживает годы, группы, учётные записи, готовит смену года и следит за целостностью сведений.

На сервере задействованы ORM и контроллеры Django [20]; в браузере — частично обновляемые фрагменты и асинхронные запросы [21]. Там, где нужен обмен без полной перезагрузки страницы в режиме диалога, задействован стек Channels поверх WebSocket [22, 9, 3].

Глава фиксирует место «КУРС» в организации выпускных работ, перечень ролей и то, как клиент и сервер стыкуются в общей схеме.

1.10 Требования и сценарии использования

В этом разделе фиксируются требования к поведению системы в терминах пользователей и типовых действий. Такой формат удобен тем, что каждое требование можно проверить напрямую на сценарии: либо оно выполняется шаг за шагом, либо нет.

Роли и ожидаемые действия пользователей. В системе выделены три роли: студент, преподаватель и администратор. Сводно их цели и допустимые операции приведены в таблице 1.4.

Таблица 1.4: Роли пользователей и основные действия в системе

Роль	Цель	Ключевые действия
Студент	Выбрать тему и вести комплект документов	Просмотр каталога тем; подача заявки; отслеживание статуса решения; загрузка/обновление документов по типам; просмотр обсуждения по теме; получение печатных форм (где доступно)
Преподаватель	Управлять темами и принимать решения по заявкам	Создание/редактирование тем; включение/отключение видимости темы; просмотр очереди заявок; принятие/отклонение заявки с фиксацией решения; коммуникация со студентом по утверждённой теме
Администратор	Поддерживать учебный цикл и целостность данных	Управление учебными годами и группами; контроль учётных записей и ролей; административное сопровождение тем; операции подготовки/импорта данных; контроль корректности справочников и связей

Нефункциональные требования, проверяемые через сценарии. При выполнении пользовательских цепочек система должна обеспечивать предсказуемость действий: понятные статусы, корректные сообщения об ошибке, сохранение результатов операций и отсутствие «тихих» сбоев при повторении шагов. Для эксплуатации в учебный период также критичны воспроизводимость стенда и устойчивость к типовым пиковым нагрузкам (массовый выбор тем, массовые загрузки документов).

Сценарии использования. Ниже приведены типовые сценарии, на которые опиралась разработка и по которым затем выполнялось покрывающее тестирование.

Сценарий 1: вход в систему и определение роли.

- пользователь вводит логин и пароль;
- система выполняет проверку через LDAP и создаёт/обновляет локального пользователя;
- на основе профиля отображаются доступные разделы: кабинет студента, кабинет преподавателя или административный контур.

Сценарий 2: просмотр каталога тем и фильтрация.

- студент открывает каталог тем;
- применяет фильтры (например, по направлению/кафедре/курсу, если они доступны в интерфейсе);
- выбирает карточку темы и знакомится с описанием и ограничениями по вместимости.

Сценарий 3: подача заявки на тему.

- студент отправляет заявку на выбранную активную тему;
- система создаёт запись заявки со статусом ожидания и отражает её в интерфейсе студента;
- тема продолжает быть доступной другим студентам, пока не исчерпана вместимость и пока заявка не принята.

Сценарий 4: решение по заявке преподавателем.

- преподаватель открывает список заявок по своим темам;
- принимает или отклоняет заявку;
- система фиксирует решение, обновляет статусы и связывает утверждённую тему с записью обучения студента.

Сценарий 5: загрузка и обновление документов студентом.

- студент выбирает тип документа (промежуточный отчёт, презентация, итоговые материалы и т.п.);
- загружает файл; система сохраняет его и фиксирует время загрузки;
- при повторной загрузке того же типа документ обновляется без появления дубликатов.

Сценарий 6: просмотр реестра работ и формирование отчёта.

- пользователь с правами просмотра открывает реестр работ;
- применяет фильтры (год, группа, кафедра и др.);
- формирует печатную форму по текущей выборке.

Сценарий 7: обсуждение по теме (коммуникация).

- преподаватель и студент переходят в обсуждение по утверждённой теме;
- сообщения сохраняются и отображаются обеим сторонам;
- доступ к обсуждению ограничен участниками процесса (преподаватель темы и студент с принятой заявкой).

Сценарий 8: административное сопровождение учебного года и данных.

- администратор проверяет актуальность учебного года и групп;
- при необходимости выполняет импорт/подготовку данных для очередного цикла;
- контролирует корректность ролей и прав доступа без ручных правок в содержимом пользовательских записей.

Глава 2

Реализация и тестирование модернизации

2.1 Исходная архитектура и база данных

На первом этапе архитектура строилась вокруг базового контура учёта: учебный год, группа, студент, запись обучения и документ [5, 4]. Такая модель позволила перенести в Django основную прикладную логику и отказаться от прямой зависимости от файловой структуры как единственного источника истины. Проектирование было направлено на достижение минимально достаточной нормализации, чтобы обеспечить целостность данных и удобство администрирования.

Архитектура первой части формировалась по принципу «стабильное ядро + расширяемые сценарии». Это означало, что базовые сущности и связи должны оставаться неизменными при добавлении новых функций, а прикладные процессы — наращиваться поверх ядра без разрушения существующих данных. Такой принцип особенно важен для образовательных систем, где накопленные сведения по прошлым годам имеют длительный срок актуальности и должны быть доступны для отчётности.

В качестве прикладной модели была использована классическая серверная схема: запрос обрабатывается представлением, которое обращается к ORM-слою, получает/изменяет данные и формирует ответ для шаблона [6, 20]. На первом этапе этого было достаточно для закрытия основного объёма задач. Одновременно в проектировании закладывалась возможность дальнейшего перехода к смешанной модели, где часть операций выполняется асинхронно и обновляет интерфейс без полной перезагрузки страницы [3, 21].

Общая архитектурная схема решения приведена на рисунке 2.1.

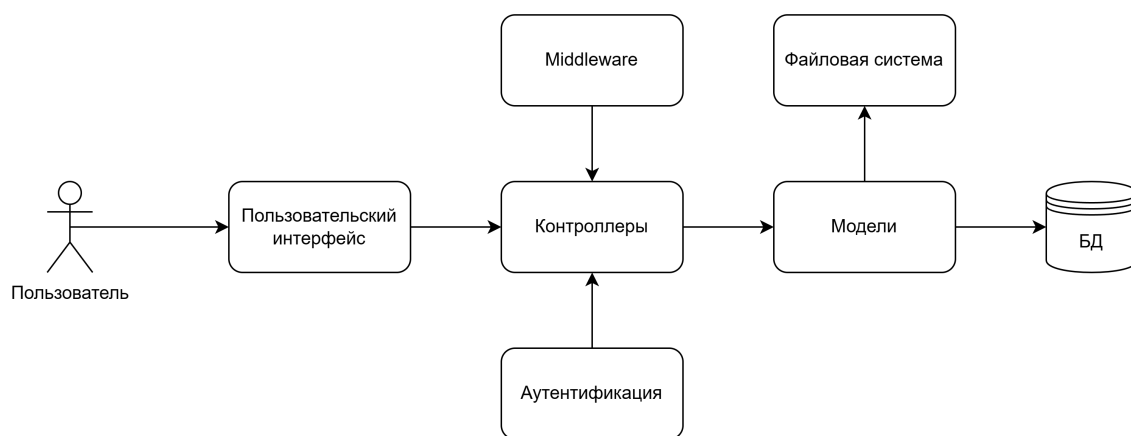


Рис. 2.1: Архитектура системы

2.2 Изменения в проектировании

Во второй части архитектура была расширена за счёт новых бизнес-сущностей и ролевых надстроек. В модель добавлены темы, заявки на темы, профили преподавателя и администратора, а также контур обсуждений. Эти изменения потребовались для перехода от простого хранения работ к управлению полным жизненным циклом выбора темы и коммуникации участников. В результате структура БД стала более выразительной по отношению к предметной области, а проектирование — ориентированным на масштабируемое развитие функционала.

Изменение структуры данных выполнялось по принципу обратной совместимости: новые сущности встраивались так, чтобы существующие записи первой части сохраняли корректность и могли участвовать в новых сценариях без ручной реконструкции. Это особенно важно для учебных систем, где данные по нескольким выпускам должны оставаться доступными в общем контуре поиска и отчётности. По этой причине в проектировании акцент сделан на явных связях, контроле уникальности и управляемом каскаде операций.

Отдельное внимание уделялось ролевой модели. В отличие от базового контура, где основной акцент делался на хранении и поиске данных, расширенная часть потребовала чётко разделить полномочия преподавателя, студента и администратора. Это повлияло на структуру таблиц и на правила доступа в серверной логике: одни и те же сущности должны обслуживать разные сценарии, сохраняя единый источник истины и предсказуемое поведение интерфейса.

С архитектурной точки зрения именно во второй части проектирования был закреплён переход от «архива документов» к «системе управления учебным процессом по темам». Таблицы стали отражать не только факт хранения файла, но и состояние процесса: публикация темы, подача заявки, принятие решения, коммуникация сторон и последующее сопровождение записи обучения. Это повысило информативность системы и уменьшило объём ручных согласований вне платформы.

2.3 Общая архитектура данных

Хранилище спроектировано как набор связанных таблиц; прикладной доступ идёт через Django ORM [5, 20]. На машине разработчика удобен SQLite, а в бою — PostgreSQL. Ядро схемы отражает учебную реальность: год обучения, группа, студент, запись о зачислении (enrollment), тема, заявка и прикрепленные к работе файлы.

Связи задаются штатными полями Django: внешние ключи для связей «многие к одному», связь один-к-одному для надстройки учётной записи профилями ролей [4, 5]. Так выдерживаются ограничения целостности и не плодятся копии одних и тех же сведений: сведения о дипломе сосредоточены в записи обучения и обновляются по мере согласования темы и появления новых файлов.

2.4 Диаграмма связей между таблицами

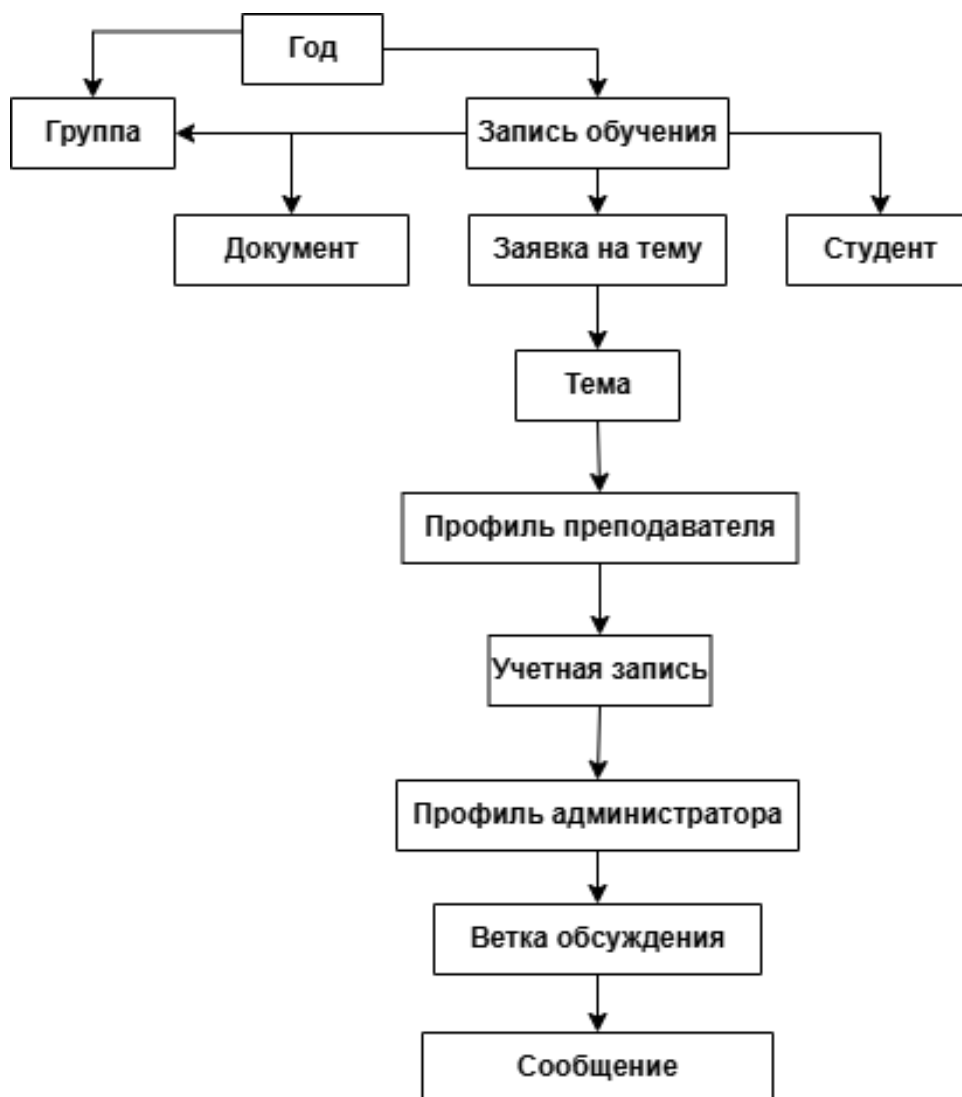


Рис. 2.2: Схема основных таблиц и связей «КУРС»

Ключевые связи между сущностями для ролевых и документных сценариев показаны на рисунке 2.2.

2.5 Описание основных таблиц

Year (учебный год). Таблица нужна для фиксации учебных периодов и разделения данных по годам.

Group (группа студентов). Таблица создана для хранения академических групп в разрезе учебного года и для привязки студентов к потоку.

Student (студент). Таблица хранит базовый справочник студентов, используемый во всех учебных сценариях системы.

Enrollment (запись обучения). Таблица объединяет студента, группу и учебный год и служит центральной точкой для данных о теме и руководителе.

Document (документ). Таблица предназначена для учёта загруженных файлов работ и фиксации типа документа и времени загрузки.

TeacherProfile (профиль преподавателя). Таблица введена для хранения профиля преподавателя и связи учётной записи с учебными процессами в системе.

Topic (тема). Таблица создана для публикации и сопровождения тем работ, доступных студентам для выбора.

TopicRequest (заявка на тему). Таблица нужна для процесса согласования темы между студентом и преподавателем с хранением статуса решения.

AdminProfile (профиль администратора). Таблица выделена для назначения административной роли и управления темами на уровне кафедры.

DiscussionThread и DiscussionMessage (обсуждения). Эти таблицы формируют контур переписки по теме: контейнер обсуждения и сообщения участников.

auth_user (учётная запись Django). Таблица является базовым источником данных авторизации и точкой привязки ролевых профилей приложения.

Схема балансирует между нормализацией и удобством выборок для экрана; ограничения в БД повторяют правила предметной области, а опасные удаления блокируются или каскадируются осознанно. В этой части также фиксируются сущности, необходимые для ролевой модели, обсуждений и корректного отображения сроков по документам.

2.6 Реализация функциональности и принятые решения

В первой части проекта была реализована рабочая версия системы с обязательными сценариями учебного процесса: ведение карточек студентов и записей обучения, загрузка документов, фильтрация работ и формирование отчётов. Этот слой сформировал устойчивую основу, подтвердил корректность выбранного стека и позволил перейти от исследовательского этапа к практической эксплуатации.

Базовая реализация проектировалась как эксплуатационный минимум, достаточный для повседневной работы кафедры. В практическом смысле это означало, что пользовательские цепочки должны быть завершёнными: пользователь входит в систему, находит нужный раздел, вносит данные, получает подтверждение и может вернуться к задаче позже без потери контекста. Реализация именно таких сквозных сценариев стала приоритетом первой части.

Технически базовый этап зафиксировал стандарты, которые затем использовались во второй части: единый стиль маршрутизации, согласованная работа моделей и представлений, предсказуемые правила хранения файлов и структура шаблонов, пригодная для дальнейшего расширения интерфейса. Благодаря этому расширение функционала происходило как развитие, а не как переписывание существующего ядра.

Реализация расширений.

Вторая часть развития опиралась на уже действующий функционал и дополняла его новыми сценариями без нарушения совместимости с ранее накопленными данными. Ключевые решения были связаны с управлением темами и заявками, реализацией ролевой навигации, внедрением контура обсуждений и обеспечением согласованности данных при операциях импорта, загрузки и обновления записей.

Реализация расширений велась с учётом реальных точек нагрузки в учебном процессе: период массового выбора тем, интенсивная коммуникация между студентом и руководителем, частые обновления статусов и необходимость оперативной отчётности. Поэтому во второй части акцент был сделан не только на добавление новых экранов, но и на согласованность всего цикла данных — от первичного ввода до итогового отчёта.

Отдельный слой решений касался удобства сопровождения. Расширенный функционал должен был оставаться управляемым для администрации кафедры без постоянного вмешательства разработчиков. Это потребовало продуманной роли администратора, контроля корректности операций удаления/скрытия и явных ограничений на действия, способные нарушить процесс согласования тем. Такой подход снизил операционные риски и сделал систему пригодной для регулярной работы в течение семестра.

Ключевые реализованные подсистемы.

В одном контуре «КУРС» совмещены календарь учёбы и группы, конвейер тем и заявок, файловое хранилище, реестр работ с фильтрами и обсуждения по теме в режиме, близком к реальному времени.

Модель данных и роли. Наличие `AdminProfile` открывает кураторскую админ-панель; право выдаётся из Django-админки. У `Topic` используется признак `is_active`: скрытие темы блокируется, пока существуют активные заявки. Для `Enrollment` допускается пустая кафедра с корректным отображением «Не указана».

Аутентификация и навигация. Вход организован через LDAP-модуль: запись пользователя проверяется, локальный пользователь создаётся или обновляется, а роль определяется профилем. В `middleware` к запросу подмешиваются профили, что обеспечивает единообразие навигации и отображения сценариев под роль.

Пользовательские сценарии. Реализованы: поиск и просмотр работ с фильтрами, админский CRUD тем, кабинет преподавателя с обработкой заявок, а также операции загрузки и удаления документов с синхронизацией индекса материалов.

Отчётность и коммуникации. Печатные формы формируются через WeasyPrint [24]; обсуждения реализованы через Channels/WebSocket с проверкой прав доступа (преподаватель темы и студенты с принятой заявкой) [22, 23, 9].

Развёртывание на тестовой среде (Docker Compose).

Стенд поднимается через Docker Compose: PostgreSQL, Redis, приложение и Nginx [25, 23].

Переменные окружения. Читаются из окружения (на тесте обычно из локального конфигурационного файла рядом с кодом):

- секрет, флаг отладки, белый список хостов;
- реквизиты PostgreSQL: база, логин, пароль, хост, порт.

Для сокетов отдельно задаются параметры Redis-backend каналов: включение, хост, порт, номер БД.

Запуск. Собираются образы, стартуют СУБД, кэш, приложение и прокси. При старте контейнера приложения выполняется импорт наследуемых проектов, `collectstatic` и запуск Daphne под ASGI.

Nginx отдаёт статику и медиа с томов, проксирует HTTP к приложению и корректно поднимает WebSocket.

Глава охватывает пользовательские и интеграционные возможности «КУРС», LDAP, индекс материалов, чаты, PDF и контейнерный запуск, плюс условное разделение направлений между соавторами.

Тестирование на тестовой среде.

Тестирование мы строили как непрерывную проверку всей системы, а не как формальный список отдельных запусков [1]. Сначала подтвердили, что базовый контур работает стабильно (профили, регистрация, документы, поиск), затем проверили расширения: темы, заявки, административные операции, обсуждения через WebSocket и импорт унаследованных данных. Мы использовали позитивные покрывающие сценарии: каждый сценарий повторяет реальную пользовательскую цепочку и должен выполняться до конца без ручного вмешательства.

Тестовый стенд мы собрали в той же конфигурации, что и рабочий контур: приложение, база, прокси и Redis для realtime-канала. Это дало возможность увидеть не только поведение отдельных экранов, но и согласованность всей цепочки: как данные пишутся в БД, как отражаются статусы в интерфейсе, как формируются отчёты и как ведёт себя чат при одновременной работе пользователей.

Перед каждым прогоном мы фиксировали исходное состояние: выбранный учебный год, набор тестовых пользователей, предварительно загруженные данные и ожидаемые

статусы. Такой порядок убрал двусмысленность при анализе результата. Если тест проходил успешно, мы получали не просто отметку «ОК», а понятное подтверждение, что конкретный шаг работает так же, как его выполняет пользователь в реальном процессе.

План тестирования и принципы покрытия. Тестирование было организовано в виде набора проверяемых цепочек с единым подходом к подготовке среды и фиксации результата.

- **Состав стенда:** приложение, PostgreSQL, Nginx и Redis (для realtime-канала), что соответствует эксплуатационному контуру.
- **Набор тестовых пользователей:** студенты и преподаватели с корректными LDAP-учётными записями и преднастроенными профилями, а также пользователь с административными правами.
- **Данные для прогона:** учебный год, минимум две группы, несколько тем с разной вместимостью, заявки в разных статусах и набор документов по ключевым типам.
- **Критерий успеха:** сценарий проходит до конца без ручного вмешательства, а итог отражается одинаково в интерфейсе и в данных (статусы, связи, файлы).
- **Контроль повторяемости:** часть сценариев выполнялась повторно (в том числе с обновлением документов и повторной подачей заявки), чтобы исключить эффект одноразовой удачи.

Позитивные покрывающие тесты системы.

Таблица 2.1: Позитивные покрывающие тесты в тестовой среде

Тестовый сценарий	Ответственный	Ожидаемый результат	Фактический результат
Авторизация и открытие личного кабинета студента	Пянтин А.С.	Успешный вход, корректная роль и доступные разделы	Выполнено успешно, права и меню соответствуют роли
Создание и сохранение карточки темы преподавателем	Пянтин А.С.	Новая тема появляется в списке и доступна для выбора	Выполнено успешно, тема создана и отображается в каталоге
Подача заявки студентом на активную тему	Пянтин А.С.	Заявка сохраняется со статусом ожидания, счётчик обновляется	Выполнено успешно, статус и счётчики обновлены корректно
Решение по заявке преподавателем (принятие)	Пянтин А.С.	Статус заявки меняется на принятый, тема фиксируется в записи обучения	Выполнено успешно, решение отражено в интерфейсе и в данных
Загрузка промежуточного отчёта и проверка отображения	Титов Е.В.	Документ загружается, ссылка и дата загрузки отображаются в интерфейсе	Выполнено успешно, файл доступен по ссылке, метка времени сохранена
Фильтрация реестра работ по году, группе и кафедре	Титов Е.В.	Таблица возвращает корректный набор строк без лишних записей	Выполнено успешно, критерии фильтрации применяются без искажений
Открытие обсуждения темы и отправка сообщения	Титов Е.В.	Сообщение сохраняется в истории и отображается обоим участникам	Выполнено успешно, сообщение видно в истории диалога обеих сторон
Формирование PDF-отчёта по результатам поиска	Титов Е.В.	Генерируется файл отчёта с актуальными данными выборки	Выполнено успешно, PDF сформирован и содержит корректные записи

Подробный перечень покрывающих сценариев и фактических результатов представлен в таблице 2.1.

Расширенный набор тестовых сценариев. Чтобы убедиться, что система корректно ведёт себя не только в «идеальном» прогоне, мы отдельно проверили типовые пограничные ситуации. Для входа и ролевой навигации смотрели повторный вход одним пользователем, вход преподавателя (без студенческих атрибутов) и запрет неавторизованного просмотра каталога тем. Для контура заявок проверяли запрет подачи заявки на неактивную тему, защиту от дубликатов, а также ситуацию конкуренции при вместимости темы = 1: в таком случае принимается только одна заявка, а остальные получают корректное состояние и решение.

В действиях преподавателя дополнительно проверили отклонение и принятие заявки

с фиксацией статуса, блокировку скрытия темы при наличии активных заявок и допустимость скрытия при их отсутствии. Для операций с темами отдельно прогнали удаление темы с очисткой связанных полей у студентов там, где это предусмотрено сценариями сопровождения.

В блоке документов проверили отказ на неверный тип файла, обновление документа того же типа без появления дубликатов, корректное удаление ранее загруженного файла и синхронизацию отображения (включая индекс материалов). Для реестра и отчётности отдельно прогнали фильтры по учебному году и группе, а также формирование PDF по текущей выборке. Для подсистемы обсуждений проверили запрет доступа без принятой заявки, корректный доступ по принятой заявке, сохранение и отображение сообщений, а также параллельную отправку сообщений без потерь и нарушений доставки.

Наконец, для административного сопровождения прогнали импорт архивных данных в режиме dry-run (без записи) и в режиме записи, контролируя создание сущностей и корректность связей. Дополнительно проверили, что доступ к административному разделу приложения появляется только при наличии административных прав.

Таблица 2.2: Краткая выборка расширенных тестовых сценариев

Сценарий	Ожидаемый результат	Фактический результат
Подача заявки на неактивную тему	Заявка не создаётся, отображается ограничение	Выполнено, заявка не создана
Повторная заявка на ту же тему	Дубликат не создаётся	Выполнено, дубликатов нет
Скрытие темы при активной заявке	Операция блокируется	Выполнено, блокировка работает
Перезагрузка документа того же типа	Документ обновляется без дублей	Выполнено, обновление корректно
Открытие обсуждения без принятой заявки	Доступ запрещён	Выполнено, доступ закрыт
Импорт архивных данных (dry-run)	План импорта без записи в БД	Выполнено, без записи

Мы разделили тесты поровну между участниками: по четыре сценария на каждого, с покрытием и серверной, и пользовательской логики. После полного прогона получили восемь успешных результатов из восьми; критических отклонений не выявили. Небольшие замечания по интерфейсным деталям исправили сразу, до финальной фиксации отчёта.

Главный практический итог тестирования в том, что весь пользовательский цикл проходит без разрывов: вход в систему, навигация по роли, выбор темы, обработка заявки, загрузка файла, формирование отчёта и переписка в обсуждениях. Данные между экранами не теряются: статусы, ссылки и карточки отображаются согласованно.

Для нас это означает, что система готова к типовой работе кафедры в тестовом контуре: базовые функции и новые расширения не конфликтуют, а следующую итерацию развития можно выполнять поэтапно, опираясь на уже проверенную основу.

Отдельно мы проверили повторяемость результатов: запускали ключевые сценарии несколько раз на разных данных, чтобы убедиться, что поведение не зависит от случайных факторов. Это важно для реальной эксплуатации, потому что в течение семестра объём записей меняется, а система должна сохранять стабильность и предсказуемость.

Глава 3

Результаты модернизации

3.1 Сравнение с предыдущей версией системы

В этом разделе сопоставляется текущая реализация «КУРС» (Python/Django) и предыдущая версия системы [4, 7, 1]. Чтобы не дробить выводы на несколько коротких подпунктов, основные различия собраны в таблице, а затем дополнены пояснениями и итоговым выводом.

Количественные показатели модернизации приведены в таблице 3.1; качественные отличия — в таблице 3.2.

Таблица 3.1: Количественные характеристики предыдущей и текущей версий «КУРС»

Показатель	Предыдущая версия	Текущая версия
Общий объём разработки (LoC)	1830	4432
Число прикладных таблиц в БД (без <code>auth_user</code>)	0 (файловая модель)	11
Число ролей в прикладной модели	2 (студент, прочие)	3 (студент, преподаватель, администратор)
Контур «заявка–решение»	нет	да
Встроенное обсуждение по теме	нет	да (WebSocket)
Позитивных покрывающих тестов на стенде	—	8
Сервисов в контейнерном стенде	1–2 (монолит/WAR)	4 (приложение, PostgreSQL, Redis, Nginx)

Основные различия между версиями системы сведены в таблицу 3.2.

Таблица 3.2: Расширенное сравнение предыдущей и текущей версий системы

Ключевой аспект	Предыдущая версия	Текущая версия
Технологическая основа	Ring/Compojure, Selmer/Hiccup, сборка через Leiningen и WAR-деплой	Django, ORM, ASGI/Daphne, контейнерный запуск через Docker Compose
Единица хранения данных	Каталог студента + индексный файл (<code>index.xml/json/edn</code>)	Нормализованные таблицы (<code>Year</code> , <code>Group</code> , <code>Student</code> , <code>Enrollment</code> , <code>Document</code> , <code>Topic</code> , <code>TopicRequest</code>)
Контроль целостности	Логика корректности в парсерах и правилах работы с ФС	Ограничения на уровне моделей (уникальные пары, внешние ключи, <code>on_delete</code> , выборки ORM)
Аутентификация	LDAP + тестовый вход с «мастер-паролем» и служебными пользователями	LDAP backend в Django с автосозданием/обновлением локального пользователя и профилей
Ролевая модель	Базовое деление студент/прочие + админский режим через тестовый вход	Явные сущности ролей: <code>TeacherProfile</code> , <code>AdminProfile</code> , отдельные сценарии для студента/преподавателя/администратора
Работа с темами	Базовые сценарии регистрации и просмотра	Каталог тем, публикация, редактирование, скрытие/показ, востановимость темы
Контур заявок	Нет выделенного устойчивого процесса «заявка–решение»	Отдельная модель <code>TopicRequest</code> со статусами ожидания/принятия/отклонения и фиксацией решения
Коммуникация по теме	Встроенный realtime-контур отсутствует	<code>Channels</code> + <code>WebSocket</code> : <code>DiscussionThread</code> и <code>DiscussionMessage</code> ; проверка прав на подключение, рассылка сообщений в группу
Работа с документами	Загрузка и удаление файлов в структуре каталогов, опора на индекс	Типизированные документы (<code>doc_type</code>), единые правила пути хранения, уникальность документа по типу в рамках записи обучения
Импорт исторических данных	Прямая работа с архивом на файловой системе	Управляемая команда <code>import_legacy_projects</code> с разбором <code>index.clj</code> , синхронизацией с LDAP и переносом документов в модели
Эксплуатационный контур	Ориентация на WAR/Tomcat и файловую дисциплину	Контейнерная схема (приложение, PostgreSQL, Redis, Nginx), лучше воспроизводимость тестовой/рабочей среды

Дополнительные сравнительные наблюдения.

Если смотреть на систему глазами пользователя кафедры, главное изменение очевид-

но: раньше это был файловый архив с веб-интерфейсом, теперь — процессная система с явно заданными сущностями и состояниями. В старой версии всё держалось на каталогах и индексах; в новой ключевые связи между студентом, темой, заявкой и документами фиксируются на уровне модели данных.

Практический эффект этого перехода — в устойчивости. В файловой модели корректность зависела от дисциплины: как названы файлы, где лежат папки, одинаково ли заполнены индексы. В реляционной модели значительная часть этих проверок формализована: ограничения, внешние ключи и валидация уменьшают число скрытых ошибок.

Отдельное различие видно в ролевом контуре. В предыдущей Clojure-версии роль преподавателя в основном выражена в специальных страницах поиска и в особенностях LDAP-авторизации, а администраторские права могли включаться через тестовый режим. В текущей системе роли разделены предметно: студент, преподаватель и администратор имеют собственные сценарии и ограничения, а админский доступ закреплён через отдельный профиль `AdminProfile`. Для реальной эксплуатации это критично, поскольку снижает риск случайного смешения прав и делает поведение интерфейса предсказуемым.

Существенно расширился процесс работы с темой. В старом контуре тема, по сути, фиксировалась в данных работы и отражалась в поисковых/отчётных операциях. В новой реализации тема является самостоятельной управляемой сущностью с параметрами, вместимостью и состоянием активности. Поверх неё построен явный цикл заявок: студент подаёт заявку, преподаватель принимает или отклоняет, решение фиксируется, а связанное состояние записи обучения обновляется без ручной синхронизации. Именно этот контур закрывает основной организационный разрыв, который раньше приходилось решать перепиской и внешними таблицами.

Новая версия также меняет характер коммуникации. В Clojure-реализации нет выделенной realtime-подсистемы для обсуждения темы; общение остаётся внешним по отношению к системе. В Python-версии используется `Channels` и `WebSocket`, обсуждение связано с темой и проходит через `DiscussionThread/DiscussionMessage`. Дополнительно реализована проверка доступа: к каналу допускаются только преподаватель темы и студент с принятой заявкой. Это превращает чат из вспомогательного инструмента в часть управляемого учебного процесса.

Заметное отличие есть и в стратегии миграции/наследования данных. Ранее работа шла непосредственно по архиву, где данные и документы были изначальным источником. В новой системе предусмотрена отдельная команда `import_legacy_projects`, которая читает `index.clj`, синхронизирует имена с LDAP, создаёт/обновляет сущности БД и переносит документы в типизированные записи. Такой механизм снижает стоимость перехода между версиями и позволяет не терять накопленную историю при архитектурном обновлении.

В эксплуатационном плане переход к контейнерному запуску (приложение, БД, Redis, Nginx) даёт более повторяемую среду. Для учебного проекта, который сопровождается несколькими итерациями и разными составами команды, это практическое преимущество:

проще воспроизвести стенд, проще прогнать тестовые сценарии, проще диагностировать проблему на уровне отдельного сервиса. В старом подходе часть особенностей развёртывания была завязана на контур WAR/Tomcat и локальные организационные настройки.

В итоге новая версия отличается не только набором функций, но и способом развития. В старой реализации одно изменение часто затрагивало парсинг и файловую логику сразу в нескольких местах; в текущей версии доработки обычно локализуются в конкретных модулях. Это заметно упрощает следующие итерации.

Общий вывод по сравнению версий.

Сравнение показывает: переход от предыдущей Clojure-версии к Django-версии — это не косметическое обновление, а смена архитектурной основы. Первая система хорошо закрывала этап цифровизации, но при росте нагрузки и числа сценариев файловый подход стал узким местом.

Новая версия снимает эти ограничения за счёт формализованной модели, явных процессов и проверяемых правил. Больше всего это заметно в управлении темами, заявками и связью между статусами и документами. Отдельно важны realtime-коммуникация и управляемый импорт исторических данных: благодаря этому система пригодна и для текущей работы, и для преемственного развития.

С инженерной стороны Python/Django-версия ведёт себя предсказуемее: её проще расширять, передавать новой команде и разворачивать в воспроизводимой среде. Если коротко, старая система закрывала базовый учёт, а текущая уже работает как платформа сопровождения полного цикла ВКР.

Как модернизация изменила сопоставление с аналогами.

Сравнение с внешними аналогами, приведённое в разделе 1.2, выполнялось относительно исходной версии «КУРС» и отражало типичный кафедральный контур уровня «учёт работ и документов»: регистрация сведений, загрузка файлов, поиск и формирование печатных форм. На этом уровне «КУРС» сопоставима с другими локальными решениями и дополняет университетские репозитории и библиотечные контуры, однако уступает системам, ориентированным на процессный контроль (например, контроль этапов), и не покрывает ряд ежедневных организационных действий (формализованный каталог тем, заявки и фиксируемые решения).

В результате модернизации были внесены изменения, которые напрямую повлияли на позицию системы в сравнении с аналогами.

- **Переход к процессной модели выбора темы.** Добавлен управляемый каталог тем, где преподаватель публикует темы с параметрами (активность, вместимость и др.), а студент подаёт заявку. Решение преподавателя фиксируется в виде статуса, что превращает ранее неформальный процесс согласования (переписка, внешние списки) в наблюдаемый и воспроизводимый контур внутри системы.
- **Единый источник истины на уровне данных.** Переход от файловых соглашений к реляционной модели с ограничениями целостности уменьшил зависимость

корректности от структуры каталогов и ручной дисциплины. Это повышает надёжность ведения данных по нескольким учебным годам и упрощает сопровождение.

- **Коммуникация, привязанная к теме.** Реализован встроенный контур обсуждения по утверждённой теме с проверкой прав доступа. Для кафедральной практики это важно тем, что коммуникация фиксируется рядом с документами и статусами, а не уходит во внешние каналы.
- **Административный контур сопровождения.** Добавлены сценарии, закрывающие повседневные задачи кафедры (управление годами/группами, операции по темам и данным), что снижает трудоёмкость поддержки без участия разработчиков.
- **Управляемая миграция и воспроизводимое развёртывание.** Отдельные механизмы импорта и контейнерный стенд позволяют переносить накопленные данные и поддерживать повторяемую тестовую среду, что критично для университетской эксплуатации.

С точки зрения сопоставления с конкретными классами аналогов изменения проявляются следующим образом. По отношению к **вузовым сервисам и репозиториям** (ПГГПУ, СПбПУ, ВолГУ, ЮФУ) модернизированная «КУРС» усиливает кафедральный слой: университетские контуры удобны для архивирования и общевузовской отчётности, однако не закрывают ежедневные сценарии выбора темы и согласования. «КУРС», напротив, обеспечивает эти сценарии на протяжении семестра и может выступать источником структурированных данных для дальнейшей передачи в общеуниверситетские системы.

По отношению к **кафедральным системам этапного контроля** (ТПУ) модернизированная «КУРС» стала сравнимой по уровню процессности за счёт формализованных заявок и статусов, при этом сохраняет сильную сторону в интеграции темы, документов и коммуникации в одном интерфейсе. Потенциальной зоной дальнейшего развития остаётся более формальный контроль графика этапов (если он требуется регламентом), то есть сближение с классом систем, ориентированных на календарно-этапную модель.

Наконец, по отношению к **комплексным АИС СПО** модернизация подчёркивает специализацию: вместо широкого учёта учебного процесса система концентрируется на глубоком сопровождении выпускного контура, что даёт более детальные и удобные сценарии для кафедры при меньшей инфраструктурной сложности.

3.2 Метрики выполненной работы

В таблице приведена единая детализация по двум этапам разработки: первая часть (перенос и базовое ядро) и вторая часть (расширение функциональности). Для первой части объём работ был пересчитан по исходному коду первой итерации проекта; распределение между участниками выполнено пропорционально ранее принятому в первой части вкладу. Для второй части использованы согласованные значения текущего проекта.

Таблица 3.3: Детализация объёма работ по блокам (единица измерения — строки кода, LoC)

Блок / зона изменения	Первая часть проекта		Вторая часть проекта	
	Пянтин А.	Титов Е.	Пянтин А.	Титов Е.
Backend (серверная часть)	303	511	989	1218
Frontend (шаблоны, JS, CSS)	638	378	1292	933
Итого по первой части проекта	941	889	—	—
Итого по второй части проекта	—	—	2281	2151
Итого по backend	814		2207	
Итого по frontend	1016		2225	
Общий итог по проектам	1830		4432	
Общий итог по участнику (обе части)	3222	3040	3222	3040

Итоговые количественные показатели по этапам и участникам приведены в таблице 3.3; все числовые значения указаны в строках кода (LoC).

Заключение

В ходе работы мы последовательно модернизировали систему «КУРС»: разобрали исходную реализацию, перепроектировали ключевые части, внедрили новый функционал и проверили его на тестовом контуре. В результате из базового реестра работ получилась единая среда для тем, заявок, документов и взаимодействия участников.

Поставленная цель достигнута: система стала функционально шире и удобнее в повседневном использовании. Новые сценарии встроены в общую модель данных и в единые правила доступа, поэтому сопровождать систему в течение учебного года стало проще.

Позитивные покрывающие тесты подтвердили, что ключевые цепочки работают стабильно: ролевая навигация, обработка заявок, операции с документами, формирование отчётов и обмен сообщениями. Критических сбоев в тестовой среде мы не зафиксировали.

Сравнение с предыдущей версией показывает прикладной эффект: основные операции выполняются быстрее и прозрачнее, а проект получил запас для следующих этапов развития. Метрики по блокам подтверждают сопоставимый вклад участников и целостный результат командной работы.

Если смотреть на повседневную эксплуатацию, изменения ощущаются напрямую: преподавателю проще принимать решения по заявкам, студенту понятнее состояние своей работы, а администратор управляет учебным циклом в одном контуре без лишних ручных шагов.

Исходный код текущей реализации размещён в закрытом репозитории проекта [17].

Библиографический список использованной литературы

1. Емельянова И. Г. Программная инженерия : учебное пособие / И. Г. Емельянова, В. А. Емельянов // Москва : Форум : ИНФРА-М. – 2018. – 252 с.
2. Пятибратов А. П. Компьютерные информационные технологии : учебник для вузов / А. П. Пятибратов // Москва : Финансы и статистика. – 2009. – 768 с.
3. Рудаков И. В. Информационные системы : учебник / И. В. Рудаков, Е. Р. Федотова // Москва : Академия. – 2008. – 320 с.
4. Кузнецов С. Д. Основы баз данных / С. Д. Кузнецов // Москва : Бином. Университет. – 2008. – 484 с.
5. Голицына О. Л. Базы данных : учебник / О. Л. Голицына, Т. Л. Партыка, И. И. Попов // Москва : ФОРУМ : ИНФРА-М. – 2020. – 400 с.
6. Лаврищева К. Ю. Основы разработки программных систем : учебное пособие / К. Ю. Лаврищева // Москва : Лань. – 2007. – 304 с.
7. Андрианов В. Б. Проектирование автоматизированных информационных систем : учебное пособие / В. Б. Андрианов // Москва : Академия. – 2012. – 160 с.
8. Филонов П. П. Жизненный цикл программного обеспечения : учебное пособие / П. П. Филонов, И. М. Лаврушина // Москва : Финансы и статистика. – 2012. – 352 с.
9. Олифер В. Г. Компьютерные сети. Принципы, технологии, протоколы : учебник для вузов / В. Г. Олифер, Н. А. Олифер // Санкт-Петербург : Питер. – 2016. – 992 с.
10. Лапыгин Ю. Н. Объектно-ориентированное программирование : учебное пособие / Ю. Н. Лапыгин // Москва : Гардарики. – 2015. – 288 с.
11. Информационная система учёта и контроля выполнения этапов подготовки выпускной квалификационной работы // Электронный архив ТПУ. — URL: <https://earchive.tpu.ru/handle/11683/29452> (дата обращения: 20.01.2026).

12. Работа с сервисом «Выпускные квалификационные работы» // Пермский государственный гуманитарно-педагогический университет. — URL: <https://old.pspu.ru/it/sotrudniku/informacionnyje-sistemy-i-veb-resursy/rabota-s-servisom-vypusknyje-kvalifikacionnyje-raboty> (дата обращения: 12.02.2026).
13. Передача выпускных квалификационных работ / научных докладов // Научная библиотека СПбПУ. — URL: https://library.spbstu.ru/ru/pages/avtoram/peredacha_vypusknykh_kvalifikatsionnykh_rabot (дата обращения: 05.03.2026).
14. Выпускные квалификационные работы // Электронно-библиотечная система ВолГУ. — URL: <http://library.volsu.ru/vypusknye-kvalifikacionnyje-raboty/> (дата обращения: 18.12.2025).
15. Информация о размещении выпускных квалификационных работ на сайте // Южный федеральный университет. — URL: <https://inep.sfedu.ru/informaciya-o-razmeshhenii-vypusknykh-kvalifikacionnykh-rabot-na-sajte/> (дата обращения: 28.11.2025).
16. Система «КУРС» // Кафедра информатики и математического обеспечения ПетрГУ. — URL: <https://kurs.cs.petrstu.ru/> (дата обращения: 15.04.2026).
17. Репозиторий исходного кода проекта «КУРС» // GitHub. — URL: <https://github.com/TeWiсT/app1> (дата обращения: 01.05.2026).
18. Clojure Documentation. — URL: <https://clojure.org/> (дата обращения: 14.11.2024).
19. Python Documentation. — URL: <https://docs.python.org/> (дата обращения: 22.01.2025).
20. Django Documentation. — URL: <https://docs.djangoproject.com/> (дата обращения: 10.03.2026).
21. MDN Web Docs. — URL: <https://developer.mozilla.org/> (дата обращения: 17.02.2026).
22. Django Channels Documentation. — URL: <https://channels.readthedocs.io/> (дата обращения: 08.12.2025).
23. Redis Documentation. — URL: <https://redis.io/docs/> (дата обращения: 25.01.2026).
24. WeasyPrint Documentation. — URL: <https://doc.courtbouillon.org/weasyprint/> (дата обращения: 03.04.2026).
25. Docker Documentation. — URL: <https://docs.docker.com/> (дата обращения: 19.11.2025).